

Support client

Comment raisonner sur Q1 à Q4 – même méthode que le vrai test BCG X

1. Objectif de ce document

Ce PDF explique le **raisonnement**, pas seulement le code, pour retrouver les 4 questions du vrai test BCG X CodeSignal (Data analysis, Data collecting, Data processing, Classification) transposées sur le sujet support client. La mécanique et les pièges sont identiques au vrai test – seul le vocabulaire change. Lisez ce document **après** avoir essayé le notebook `notebook/customer_support_exercices.ipynb` sans aide, chrono en main.

2. Question 1 – Data analysis

2.1 Ce qu'on vous demande vraiment

Trois « insights » indépendants à calculer, puis à ranger dans un CSV à deux colonnes (`insight_type`, `value`). C'est le format canonique de la Q1 : peu importe le nombre d'insights, la structure de sortie ne change jamais.

```
agent = pd.read_csv(f"{DATA}/agents.csv")
ticket = pd.concat([pd.read_csv(f"{DATA}/tickets_{i}.csv") for i in range(1, 5)],
                    ignore_index=True)

average_rating = agent["rating"].mean()
pct_second_language = (agent["second_language"] != "no").mean() * 100
success_rate = (ticket["status"] == "Success").mean() * 100
```

COMMENT RAISONNER FACE À CETTE QUESTION

Découpez toujours en trois réflexes indépendants :

- **moyenne simple** -> `.mean()` sur la colonne demandée, rien de plus ;
- **pourcentage d'une condition** -> le réflexe `condition.mean() * 100` (la moyenne d'un booléen est exactement sa proportion de True) ;
- **combiner plusieurs fichiers avant de calculer** -> `pd.concat(..., ignore_index=True)`. Piège classique : calculer un taux fichier par fichier puis faire la moyenne des 4 taux – **faux** si les fichiers n'ont pas le même nombre de lignes (moyenne non pondérée). Toujours concatener d'abord.

SCHÉMA À RECONNAÎTRE

« Data analysis, plusieurs insights, CSV `insight_type/value` » → calculer chaque insight séparément sur les données déjà nettoyées/combinaisons, assembler en DataFrame à la fin, `to_csv(index=False)`.

ERREURS FRÉQUENTES

- Oublier de combiner les 4 fichiers `tickets_i.csv` avant de calculer le taux de succès.
- Confondre pourcentage (`* 100`) et proportion brute – relisez toujours l'énoncé pour savoir laquelle est attendue.
- Tolérance de l'évaluateur : 2 décimales – inutile d'arrondir vous-même dans le fichier, laissez la valeur complète.

3. Question 2 – Data collecting

3.1 Ce qu'on vous demande vraiment

Ce n'est PAS un exercice de nettoyage : c'est un exercice de **jointure + agrégation + dérivation de colonnes**. Vous recevez agents.csv, headsets.csv et 4 fichiers d'tickets, et vous devez reconstruire une seule table avec un schéma de sortie précis.

COMMENT RAISONNER FACE À CETTE QUESTION

La méthode générale, valable pour toute question de ce style :

1. **Repartez du schéma de sortie demandé**, colonne par colonne, et identifiez sa source : vient-elle directement d'une table, d'une jointure, d'un calcul simple, ou d'une agrégation sur une table d'événements ?
2. **Sommer plusieurs colonnes booléennes en une ligne** -> `df[bool_cols].sum(axis=1)` donne le nombre de True par ligne ; agrégez ensuite par agent avec `groupby().sum()`.
3. **"Nombre de jours depuis une date"** -> toujours `pd.to_datetime` puis soustraction d'une date de référence fournie par l'énoncé (ici 2023-04-15, jamais `pd.Timestamp.now()`), puis `.dt.days`.
4. **Une formule explicite donnée dans l'énoncé** (`experience = 2023 - started_year`) doit être suivie à la lettre, même si une autre méthode semble équivalente.
5. **fillna(0) après un merge sur une agrégation** – un agent sans aucun événement n'a pas de ligne dans la table d'événements, donc le merge produit un NaN ; 0 événement = 0 upvote, logiquement.

```
upvote_cols = ['clarity_upvote_given', 'politeness_upvote_given', '
    resolution_speed_upvote_given', 'follow_up_upvote_given']
ticket["n_upvotes"] = ticket[upvote_cols].sum(axis=1)
upvotes_per_agent = ticket.groupby("agent_id")["n_upvotes"].sum()

headsets["last_inspection_date"] = pd.to_datetime(headsets["last_inspection_date"])
headsets["days_since_inspection"] = (TODAY - headsets["last_inspection_date"]).dt.days
```

SCHÉMA À RECONNAÎTRE

« table de sortie avec colonnes venant de plusieurs fichiers » → partir du schéma cible, mapper chaque colonne à sa source (directe / jointure / dérivée / agrégée), assembler avec des `merge` successifs.

ERREURS FRÉQUENTES

- Les tests sont **order-agnostic** (ordre des lignes/colonnes indifférent) – ne perdez pas de temps à trier, l'énoncé le dit explicitement.
- Oublier `fillna(0)` sur `number_of_upvotes` après le merge – les agents sans événement se retrouvent avec un NaN au lieu de 0.
- Recalculer `experience` autrement que par la formule exacte donnée (`2023 - started_year`).

4 4. Question 3 – Data processing

4.1 La règle qui domine tout : pas de fuite de données

Toute statistique apprise sur les données (moyenne de remplissage, encodeur, scaler) doit être calculée **uniquement sur train**, puis appliquée telle quelle à test. Ne jamais recalculer quoi que ce soit sur test.

COMMENT RAISONNER FACE À CETTE QUESTION

Quatre étapes, quatre pièges précis – traitez-les comme des blocs indépendants :

- **a. fillna(age, moyenne)** – la moyenne vient du **train uniquement**, appliquée ensuite comme constante à test.
- **b. encodage ordinal** – `sklearn.OrdinalEncoder` garantit nativement des codes consécutifs à partir de 0. `fit` sur train seul, `transform` (jamais `fit_transform`) sur test – sinon les deux fichiers peuvent avoir des mappings incohérents entre eux.
- **c. Standard Scaling** – même règle : `fit` sur train, `transform` sur test.
- **d. binaire "A class"/"B class"** – un simple `.map({"A class": 0, "B class": 1})`, aucune fuite possible ici.

ERREURS FRÉQUENTES

Le piège des « exactement 5 décimales ». Arrondir la valeur numérique (`.round(5)`) ne suffit pas : `to_csv` n'ajoute pas de zéros de padding (1.2 reste "1.2", pas "1.20000"). Formatez explicitement la colonne en chaîne avec `s.map(lambda x: f"{x:.5f}")` avant `to_csv` – et uniquement sur la colonne demandée : un `float_format="%5f"` global sur `to_csv` reformaterait **toutes** les colonnes flottantes du DataFrame, y compris `rating`, ce qui n'est pas demandé.

SCHÉMA À RECONNAÎTRE

« *Data processing avec contrainte no leakage* » → systématiquement `fit` sur train uniquement, `transform` sur test ; toute statistique de remplissage vient du train.

5 5. Question 4 – Classification

5.1 L'objectif est asymétrique, pas un bon F1

L'énoncé dit : *maximiser le recall, en gardant la precision à un niveau relativement élevé*, classe positive = `B class` (1). Ce n'est pas « un bon score global » – c'est une contrainte précise à traduire en code.

COMMENT RAISONNER FACE À CETTE QUESTION

Deux leviers à utiliser ensemble :

1. **`class_weight="balanced"`** au moment du `fit` – le modèle pénalise plus fortement les faux négatifs sur la classe positive, donc apprend mieux à la détecter.
2. **Ajuster le seuil de décision** plutôt que le seuil par défaut de `predict()` (0.5 implicite). Balayez les seuils sur `predict_proba`, gardez celui qui maximise le recall **sous une contrainte minimale de precision** – ça traduit directement la consigne de l'énoncé en code.

Évaluez toujours sur `val.csv` (labels connus), jamais sur `test.csv` (labels absents dans le vrai test – littéralement impossible d'y calculer precision/recall en local).

```

model = LogisticRegression(max_iter=1000, class_weight="balanced")
model.fit(X_tr, y_tr)

proba_va = model.predict_proba(X_va)[:, 1]
for thr in np.arange(0.1, 0.9, 0.02):
    pred = (proba_va >= thr).astype(int)
    # garder le seuil qui maximise recall sous precision >= 0.6

```

ERREURS FRÉQUENTES

- Le bouton Run ne score que les 10 premières lignes du fichier test – ce n'est pas représentatif, ne pas paniquer si le score Run est bas ou instable ; seul Submit compte pour le score final.
- Limite d'exécution : 8 secondes – un modèle trop lourd (grid search complet, forêt de 1000 arbres) peut timeout.
- Utiliser `predict()` par défaut (seuil 0.5 implicite) au lieu de chercher le seuil qui correspond vraiment à la consigne.

SCHÉMA À RECONNAÎTRE

« maximiser recall en gardant precision élevée, classe positive minoritaire » → `class_weight="balanced"` + recherche de seuil sur `predict_proba` avec contrainte de precision minimale, validé sur le validation set.

6. Check-list finale

- ☐ Q1 : concaténer les 4 fichiers AVANT de calculer un taux global
- ☐ Q1 : `condition.mean() * 100` pour un pourcentage
- ☐ Q2 : schéma de sortie décomposé colonne par colonne avant de coder
- ☐ Q2 : `fillna(0)` après un merge sur une agrégation
- ☐ Q3 : fit sur train uniquement, transform sur test – jamais l'inverse
- ☐ Q3 : format texte à 5 décimales exactes, colonne ciblée seulement
- ☐ Q4 : `class_weight="balanced"` + recherche de seuil
- ☐ Q4 : évaluer sur val, jamais sur test (labels absents)